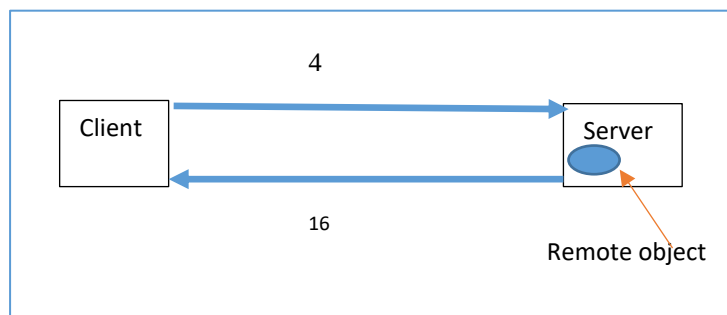
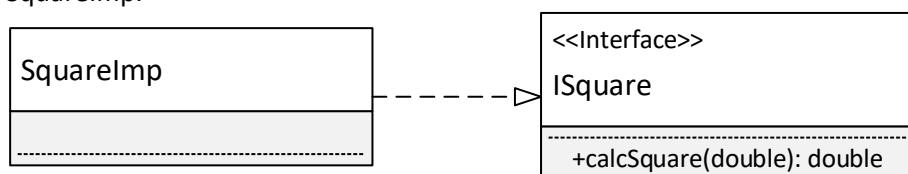


You are required to write a distributed application which consists of:



- Below is the UML diagram shows the remote interface ISquare and the implementation class SquareImp.



- ISquare** contains a method **calcSquare(double)** which returns the square of a number.

```

ISquare.java
ISquare
calcSquare(double) : double
  
```

- The class **SquareImp** implements the remote interface **ISquare**

```

SquareImp.java
SquareImp
SquareImp()
calcSquare(double) : double
  
```

- The class **ServerTester** creates, registers and binds an instance of **SquareImp**. *Call the instance obj and the reference Arithmetic; on port 2026.*

```

ServerTester.java
ServerTester
main(String[]) : void
  
```

- The class **ClientTester** calls **calcSquare(double)** with an appropriate value and displays the answer. Use the local server. A typical output is:

The square of 4 is 16

```

ClientTester.java
ClientTester
main(String[]) : void
  
```

Solution

ISquare.java

```
package rmiex1;
import java.rmi.*;
public interface ISquare extends Remote{
    public abstract double calcSquare(double numPar) throws Exception;
}
```

SquareImp.java

```
package rmiex1;
import java.rmi.*;
import java.rmi.server.*;
public class SquareImp extends UnicastRemoteObject implements ISquare {
    public SquareImp() throws RemoteException {
        super();
    }
    @Override
    public double calcSquare(double numPar) throws Exception {
        return numPar*numPar;
    }
}
```

ServerTester.java

```
package rmiex1;
import java.rmi.registry.*;
public class ServerTester {
    public static void main(String[] args) throws Exception{
        SquareImp obj=new SquareImp();//create remote object
        Registry reg=LocateRegistry.createRegistry(2026);
        reg.rebind("square", obj);
    }
}
```

ClientTester.java

```
package rmiex1;
import java.rmi.Naming;
public class ClientTester {
    public static void main(String[] args) throws Exception{
        ISquare stub=(ISquare) Naming.lookup("rmi://localhost:2026/square");//create stub
        double num=10;
        double ans=stub.calcSquare(num); call remote method
        System.out.println("The square of "+ num + " is "+ ans);
    }
}
```

Running the application

Run **ServerTester.java** then **ClientTester.java**. If there is an exception (e.g., **Port already in use: 2026; Address already in use: bind**, etc.), it means the port is in use. Remove all terminated launches ("xx") till you see the current server. Terminate the server application (click "red square")